

How Computational Thinking Skills Influence Design Quality in Scratch

An Empirical Analysis of Code Smells

Ricardo Hidalgo Aragón Jesús M. González-Barahona Gregorio Robles

Universidad Rey Juan Carlos, Madrid, Spain

MadSESE – October 2025

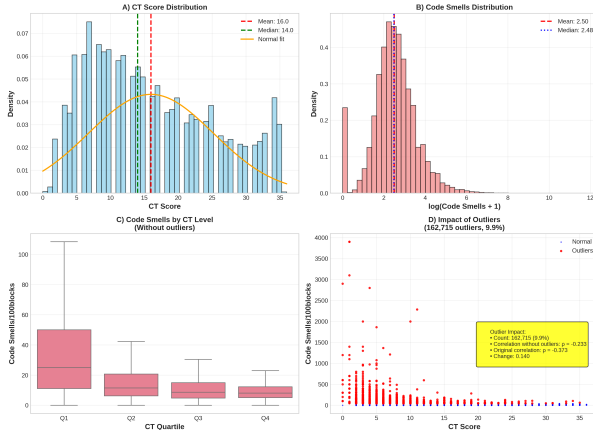
The Silent Crisis in Computing Education

The Question Nobody Asks

**Can your students make programs that work well,
not just programs that work?**

- We teach loops, conditionals, variables...
- **But, Do we teach enough code quality?**

The Silent Crisis in Computing Education



The Question Nobody Asks

**Can your students make programs that work well,
not just programs that work?**

Context: Why This Matters Now

The Old World (b.llms)

- Binary assessment: "works" vs "doesn't work"
- Focus on syntax mastery
- Code as artifact

Critical Gap

- Code smells (CS) studied in industry
- Computational Thinking (CT) models validated in education
- **But they never meet!**

The LLM Era

- GitHub Copilot, ChatGPT, Gemini...
- Programmer = **Architect + Curator**
- **Quality awareness is survival**

The Challenge

Students who can't recognize code smells won't effectively guide AI tools or refactor generated code

Research Questions

RQ1: Do CT Skills Predict Code Quality?

How do computational thinking dimensions relate to code smell density?

Hypothesis: Higher CT $\rightarrow$ Lower smell density

Research Questions

RQ1: Do CT Skills Predict Code Quality?

How do computational thinking dimensions relate to code smell density?

Hypothesis: Higher CT $\rightarrow$ Lower smell density

RQ2: Which CT Dimensions Matter Most?

Are relationships dimension-specific or uniform across CT constructs?

Hypothesis: Some CT dimensions are more protective

Research Questions

RQ1: Do CT Skills Predict Code Quality?

How do computational thinking dimensions relate to code smell density?

Hypothesis: Higher CT $\rightarrow$ Lower smell density

RQ2: Which CT Dimensions Matter Most?

Are relationships dimension-specific or uniform across CT constructs?

Hypothesis: Some CT dimensions are more protective

RQ3: Does Context Matter?

Do coding challenges moderate CT–smell relationships?

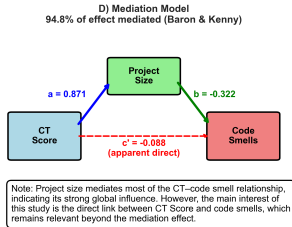
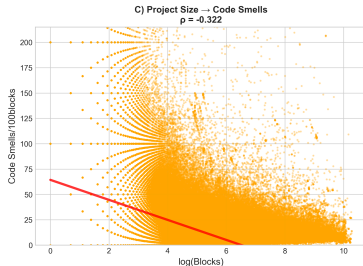
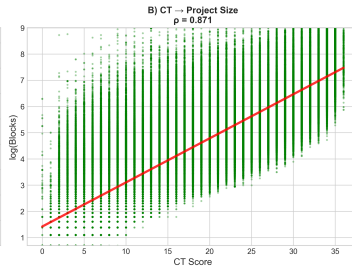
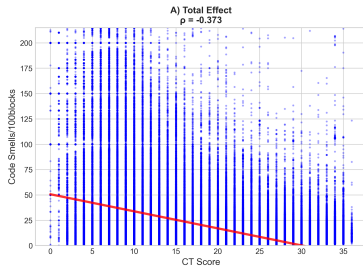
Hypothesis: Task constraints alter patterns

1,664,112

Scratch projects analyzed

CT Dimensions	Code Smells	Tools
• Abstraction • Parallelism • Logic • Synchronization • Flow Control • User Interactivity • Data Representation • Math Operators • Motion Operators	• Duplication • Long Scripts • Dead Code • Variable Scope • Message Passing • Attribute Initialization • Total of 34 CS!! • Which are grouped into 8 families	• Dr. Scratch - educational analysis • LitterBox - linter

RQ1: The CT–Quality Connection



Key Finding:

- Higher CT $\rightarrow$ Lower smell density
- **BUT: 94.8% mediated by project size**

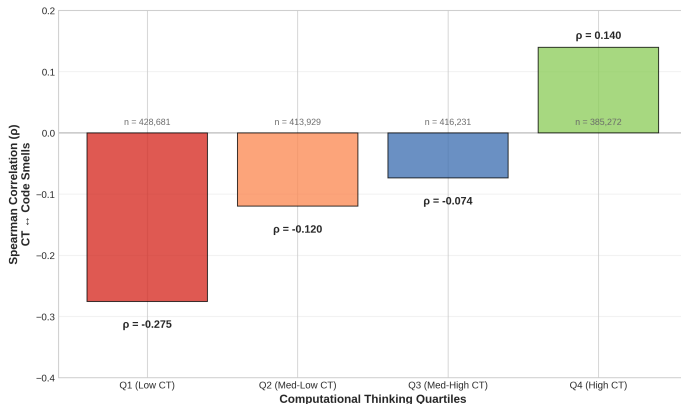
The Ambition Paradox

Skilled programmers create **larger, more complex** projects

$\rightarrow$ More opportunities for smells

$\rightarrow$ Quality improves *despite* scale

The Reversal Pattern: Q4 Paradox



Q1–Q3: CT $\uparrow \rightarrow$ Smells $\downarrow$

- Expected protective effect
- Skill reduces errors

Q4 (Largest): Pattern **reverses!**

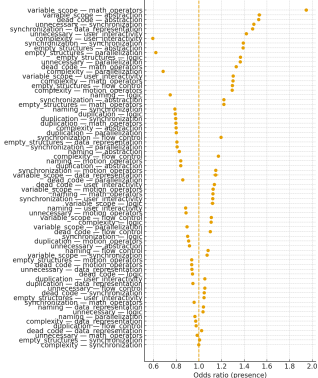
- Ambition-mediated technical debt
- Prototype $\rightarrow$ Production gap

Implication: Raw smell counts mislead in ambitious projects

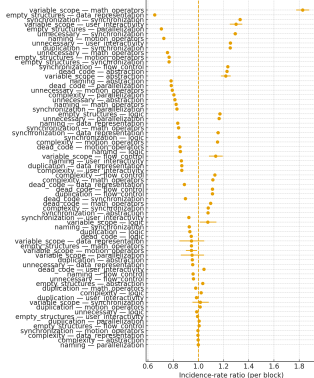
RQ2: Not All CT Dimensions Are Equal

Generalized Linear Models: 144 test (9 dimensions $\times$ 8 families $\times$ 2 outcomes)

RQ2: CT $\times$ Family Odds Ratios (logit GLM) — CT te



RQ2: CT $\times$ Family IRRs (NB-2 with exposure) — CT t



Key Patterns
(FDR-corrected, $q < 0.01$)

Abstraction & Logic: OR ≤ 1 (protect against Duplication/Dead Code)

Sync & Parallelization: OR/IRR ≈ 1.2 – 1.4 (associated with greater complexity)

Logistic (Presence):

- OR < 1 : Protective
- OR > 1 : Risk factor
- OR $= 1$: No effect

Negative B (Count):

- IRR < 1 : Reduces density
- IRR > 1 : Increases density
- IRR $= 1$: No effect

RQ3 Setup: How Do We Identify Coding Challenges?

Hybrid 4-Phase Strategy: Pattern Matching + Data-Driven Heuristics

Phase 1: Pattern Matching + Remix

- Regex on titles: "Pong", "Flappy", "Pac-Man", "Tetris" ...
- High precision, low recall
- 35,510 projects (2.1%)

Phase 2: Heuristic Enhancement

- $CT \geq 20$ (advanced)
- $Blocks \geq 100$ (complex)
- $Synchronization \geq 3$ (coordination)
- $User\ Interactivity \geq 2$ (responsive)
- +257,911 projects (17.6% total)

Phase 3: Manual Validation

- 100 random samples reviewed
- **93% accuracy**
- 7% false positives (mostly under a false project name)

Phase 4: Sensitivity Analysis

- Pattern-only vs. Heuristic-enhanced
- Fisher's z agreement: 87.5%
- Interaction $\rho = \mathbf{0.976}$ (near-perfect)
- **Same substantive conclusions**

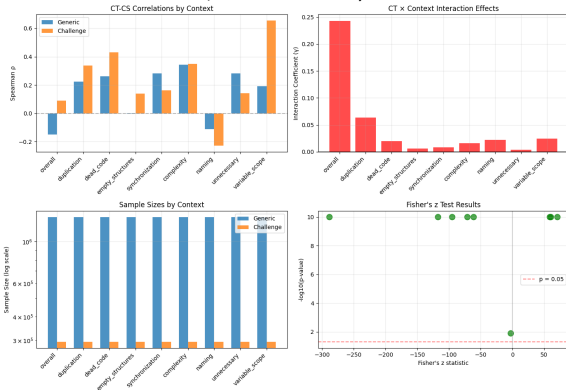
Key Insight

Heuristic approach captures $8\times$ more challenges while maintaining robust validity

→ Enables powerful moderation analysis with 293,421 challenge projects

RQ3: Context Changes Everything

RQ3: Task Context Moderation Analysis



Coding Challenges:

- Moderation effects up to $\Delta\rho = +0.37$
- Variable Scope: ρ shifts from -0.15 to $+0.22$
- **Task constraints override skill**

Context-Sensitive Smells:

- Long Scripts (51% justified)
- Variable Scope (32% justified)
- Message Passing (28% justified)

17.1% of smells seem intentional design choices

Three Mechanisms: Why Smells Appear

- ① **Skill Deficits** (expected)
 - Low Abstraction → High Duplication
 - Remediable through instruction

Three Mechanisms: Why Smells Appear

① Skill Deficits (expected)

- Low Abstraction → High Duplication
- Remediable through instruction

② Ambition-Mediated Technical Debt

- 94.8% size mediation
- Q4 reversal pattern
- Skilled programmers take on harder challenges
- → The Ambition Paradox

Three Mechanisms: Why Smells Appear

① Skill Deficits (expected)

- Low Abstraction → High Duplication
- Remediable through instruction

② Ambition-Mediated Technical Debt

- 94.8% size mediation
- Q4 reversal pattern
- Skilled programmers take on harder challenges
- → The Ambition Paradox

③ Task-Intrinsic Constraints

- Coding challenge contexts
- **Moderation effects up to $\Delta\rho = +0.37$**
- Variable Scope systematically "violates" best practices
- → Context Matters!

→ Extends software maintenance theory to education

Practical Implications: For Educators

STOP Doing This

- × Absolute smell counts
- × "Zero tolerance" policies
- × Deducting points for context-sensitive smells
- × Treating tools as interchangeable

START Doing This

- ✓ Density-based metrics
- ✓ Context-aware assessment
- ✓ Strategic sequencing (Abstraction first!)
- ✓ Normalize refactoring

Key Pedagogical Shift

From "Does it work?" to "Can it evolve?"

- Teach **maintenance literacy**: when to refactor vs when to accept trade-offs
- Recognize that 17.1% justified smell rate means **raw counts mislead**
- Quality develops gradually through identifiable stages

For Tool Developers: Adaptive Feedback

Current Static Linters:

"You have 15 smells. Bad code!"



Context-Aware Systems Should Say:

"Your variable scope pattern is *appropriate* for this challenge"

"But this duplication could be abstracted..."

"Consider refactoring when your project reaches 200 blocks"

Design Principles

- 1 Detect **project context** (challenge vs. freeform, size quartile)
- 2 Apply **taxonomy** (stable vs. sensitive vs. justified)
- 3 Provide **progressive feedback** aligned with developmental stage
- 4 Enable **opt-in complexity** ("Show me advanced patterns")

Why This Matters More Than Ever



The LLM Challenge

As AI tools (Copilot, ChatGPT) **automate code generation**, the programmer's role shifts to **architect and curator**

Students without quality awareness:

- Accept AI-generated smells
- Can't guide refactoring
- Perpetuate bad patterns

Students with maintenance literacy:

- Recognize problematic patterns
- Prompt for better designs
- Refactor strategically

Quality literacy = Professional survival skill

Computing education must cultivate

Problem Crafters

not just problem solvers

Learners who:

- Build *thoughtfully*
- Balance ambition with craft
- Write code others can understand
- Refactor when evolution demands

Educators who:

- Teach *maintenance literacy*
- Use context-aware assessment
- Normalize quality as process
- Prepare for LLM-augmented future

Thank You!

Questions?

Contact:

Ricardo Hidalgo Aragón

`r.hidalgoa.2024@alumnos.urjc.es`

Replication Package:

Full dataset, scripts & notebooks on Zenodo (MIT license) -Still pending-

Registered Report (ICSME 2025):

<https://arxiv.org/abs/2507.17314>

Backup: Detailed Methodology

Data Collection:

- 1,664,112 Scratch projects
- CT scores (DrScratch)
- Smell detection (DrScratch/LitterBox)
- Context classification

Statistical Approach:

- Robust correlation (Huber)
- Mediation analysis
- Moderation (interaction terms)
- FDR correction (Benjamini-Hochberg)

Validity Checks:

- Mutual information (construct)
- Ensemble agreement
- Heuristic challenge validation, (n=100)
- Qualitative validation (n=30)
- Construct validation (n=387)
- Sensitivity analyses

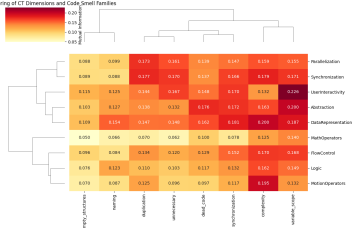
Effect Sizes:

- Spearman ρ : -0.35 to $+0.22$
- Moderation $\Delta\rho$: up to $+0.37$
- Mediation: 94.8% through size
- Cohen's κ : 0.03–0.64

All tests two-tailed, $\alpha = 0.05$, FDR-corrected

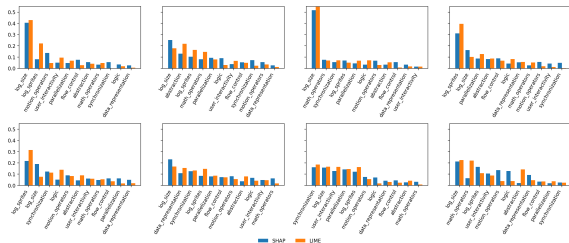
RQ2: Convergent Validation

Hierarchical Clustering of CT Dimensions and Code Smell Families



Mutual Information:

- All 9 dimensions contribute
- Similar dimensions cluster together
- Confirms structural validity



SHAP/LIME Explainability:

- Rank agreement: $\rho > 0.7$
- Validates GLM findings
- Non-linear effects detected

Triangulated Evidence

GLMs + MI + SHAP/LIME all converge:

Dimensions predict *distinct* smell families → Not reducible to “general skill”

RQ3: Testing Context-Dependence

Building on RQ1 & RQ2: Now We Ask "Does Context Change the Rules?"

What We Already Know:

RQ1: Aggregate Pattern

Higher CT $\rightarrow$ Lower smells
(but 94.8% via project size)

RQ2: Dimension-Specific

Abstraction protects (OR ≈ 0.7)
Parallelization increases (OR ≈ 1.3)

The New Question:

Do these patterns **change**
in coding challenges vs.
generic projects?

How We Test It: Interaction Model:

$$\text{Smell} = \beta_1 \cdot \text{CT} + \beta_2 \cdot \text{Context} + \gamma \cdot (\text{CT} \times \text{Context})$$

- **RQ1/RQ2:** "Does CT affect smells?"
 $\rightarrow$ Yes, via β_1
- **RQ3:** "Does this effect *differ* by context?"
 $\rightarrow$ Test via γ (interaction)
- $\gamma \approx 0$: Same rule in both contexts
- $\gamma \neq 0$: **Context changes the rule!**

Sample Split

Generic: 1,350,691 (81%)

Challenges: 293,421 (17.6%)

Analogy: Like testing if "exercise $\rightarrow$ weight loss" works differently for athletes vs. sedentary people

Qualitative Validation: The Human Check

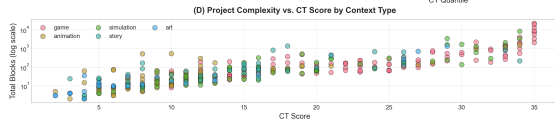
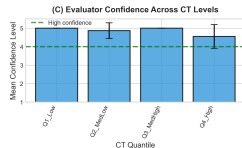
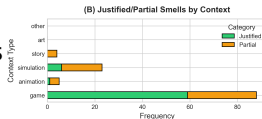
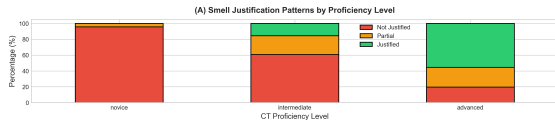
We manually inspected 387 randomly sampled projects

Findings:

- 17.1% smells justified
- Coding challenges: **>50%** for some smells
- Context-sensitive taxonomy validated

Examples:

- Long Scripts: "Boss battle logic"
- Variable Scope: "Challenge template"
- Message Passing: "Synchronization"



Critical Insight

Quantitative patterns reflect real pedagogical phenomena, not artifacts

Limitations & Future Work

Current Limitations:

- Cross-sectional (no causality)
- Scratch-specific (generalization?)
- Missing affective factors
- No demographic moderators

Causal Claims

All relationships are **associative**

Could reflect:

- Confounding (prior experience)
- Self-selection
- Reverse causation

Future Directions:

- 1 **Longitudinal:** Track CT → smell reduction over time
- 2 **Interventions:** RCTs of dimension-targeted instruction
- 3 **Mixed-methods:** Think-aloud studies
- 4 **Text-based:** Python/Java replications
- 5 **Equity:** Demographic moderators
- 6 **LLM integration:** Co-pilot + quality feedback

Replication Package

Full dataset, scripts, notebooks

Zenodo (MIT license)

Key Takeaways

- ① **CT skills influence quality**, but mediated by size (94.8%) and moderated by context
- ② **Dimensions matter differently:** Abstraction protective, Parallelization increases complexity
- ③ **Context is fundamental:** 17.1% smells justified, up to 50%+ in challenges
- ④ **New taxonomy:** Context-sensitive, context-stable, frequently justified
- ⑤ **Theoretical advance:** Three mechanisms (skill, ambition, constraints) extend maintenance theory to education

The Mission

Move from "Does it work?" to "Can it evolve gracefully?"